EXPERIMENT 10 - LAB RECORD

Performance Optimization in Zoho Creator

Efficient workflow processing in the Appointment_Booking_Form application

Submitted by	PRANAV SRINEVAS V
Register number	24101068
Application	Appointment_Booking_Form (Zoho Creator)
Subject	Low Code Application Development using Zoho Creator

1. Aim

To study performance optimization in Zoho Creator by examining a form workflow in the Appointment_Booking_Form application that sends a confirmation email, and to see how it processes only the submitted booking instead of retrieving unrelated records.

2. Introduction

Performance optimization in a low-code application means writing forms, workflows and scripts so that they use data and processing only as far as needed. Retrieving more records than required, or processing records that are not relevant to the task, wastes effort. This may not be noticeable in a small application, but it reduces efficiency as records and users grow. Efficient workflows therefore use only the data they need.

3. Application used

The practical was done in Zoho Creator on the Appointment_Booking_Form application. Deluge scripting was used in a form workflow that sends a confirmation email when a booking is submitted. The application's workflow list contains three enabled workflows, and the one studied here is **Send appoinment confirmation mail** (the name is spelled this way in the application).

Workflow	Trigger	Status
Sync_Appointment_To_Zoho_Calendar	Created > Successful form submission	Enabled
Validate Appointment Details	Created or Edited > Validations on form submission	Enabled
Send appoinment confirmation mail	Created or Edited > Successful form submission	Enabled

4. Performance optimization concept

Criteria-based fetching means retrieving only the record needed by giving a criteria, for example `Appointment_Booking_Form[ID == input.ID]`, instead of fetching every record and looping through them. The studied workflow goes one step further: it does not fetch records at all. Because the workflow runs on the form's own submission, every value it needs is already available through `input`, so only the submitted booking is processed.

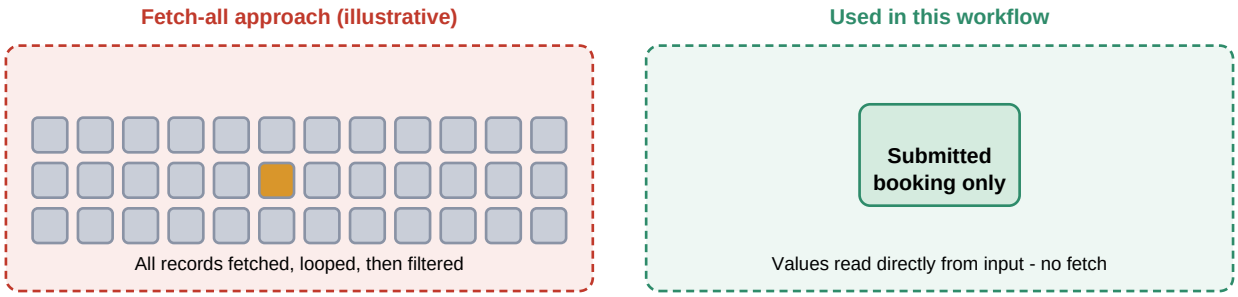


Fig. 1: Fetch-all approach compared with the approach used in this workflow (conceptual).

Illustrative comparison only (not used in the application):

```
// Inefficient: fetch every booking, then search for one
bookings = Appointment_Booking_Form[];
for each rec in bookings
{
    if(rec.ID == input.ID)
    {
        // ... process
    }
}

// Better when a record must be fetched: use a criteria
booking = Appointment_Booking_Form[ID == input.ID];
```

Approach	Steps
Inefficient	Fetch all records, iterate, find the required record, process.
Used in this workflow	Read the submitted values from input and process them directly.

5. Procedure

1. Open the workflows — Open Appointment_Booking_Form in Edit mode and go to Workflow → Form workflows. Three enabled workflows are listed for the form.

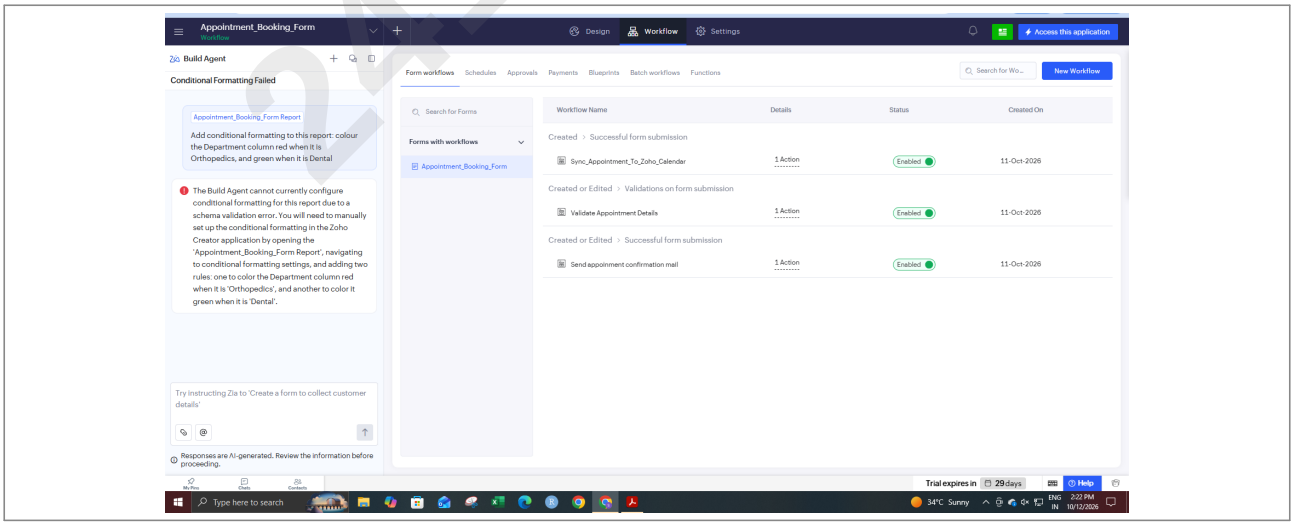


Figure 2: Form workflows of Appointment_Booking_Form

2. Open the confirmation workflow — Open *Send appoinment confirmation mail*. It runs on Created or Edited → Successful form submission, and its single action is a Deluge script.

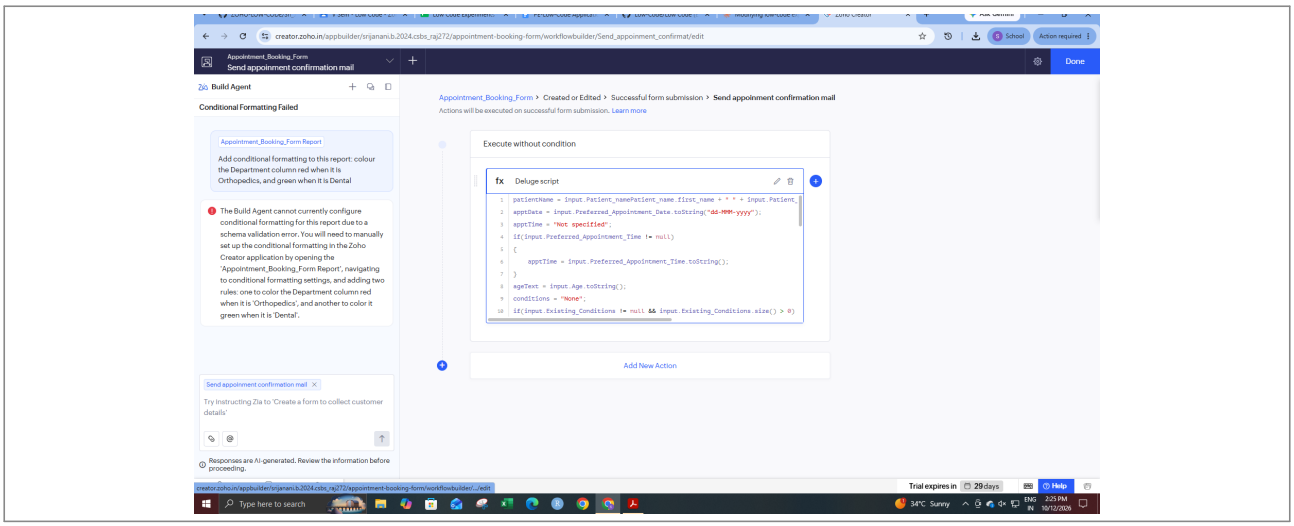


Figure 3: Workflow with its Deluge script action

3. Read the script — Open the Deluge script. It builds the patient name, date, time, age and conditions from `input`, applies defaults, checks that an email address was given, builds the message once and sends one email.



Figure 4: Deluge script of the workflow

4. Submit a booking and check the email — A booking was submitted through the form. The workflow ran and a confirmation email arrived in the inbox.

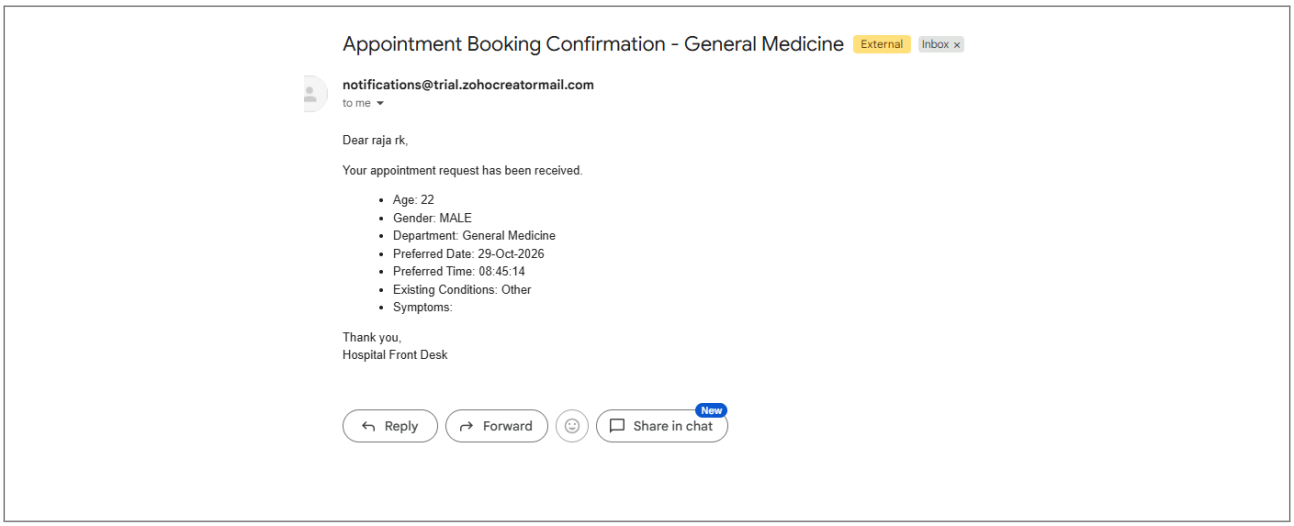


Figure 5: Confirmation email received

6. Analysis of the script

Part of the script	Why it is efficient
Values read from `input`	No record is fetched; only the submitted booking is processed.
Defaults ("Not specified", "None", `ifnull`)	Empty optional fields are handled without extra lookups or repeated checks.
Email check before sending	Mail is sent only when an address exists; otherwise a short info message is logged.
Message built once, one sendmail	A single email is composed and sent; there are no loops or repeated sends.

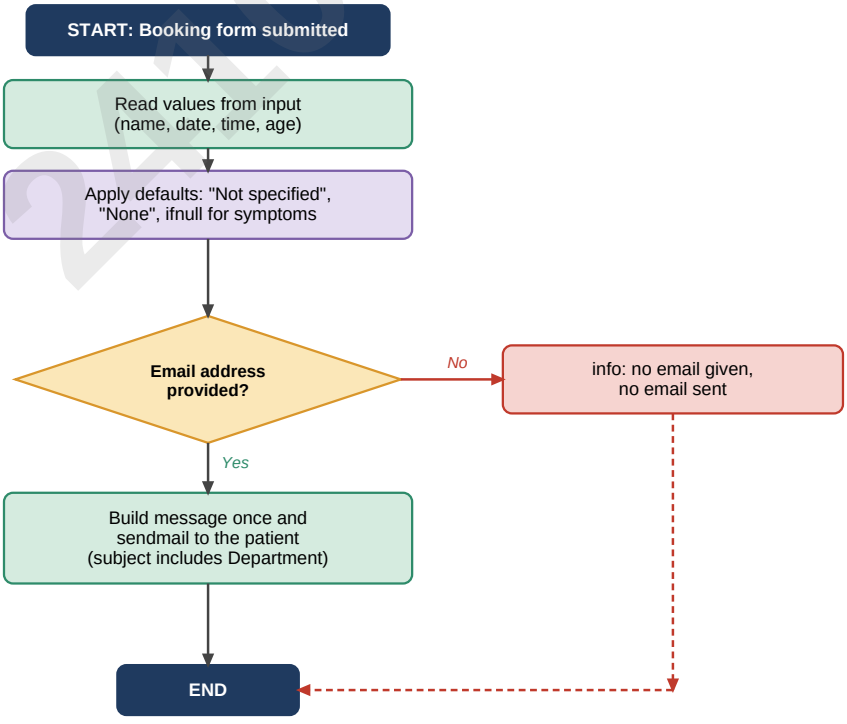


Fig. 6: Flow of the confirmation workflow.

No benchmark was performed in this practical, so no measured improvement in execution time is claimed. The comparison is based on the amount of data retrieved and processed.

7. Optimization techniques

Technique	Explanation	Status in this practical
Criteria-based record fetching	Retrieve only the records required for the operation.	Concept studied; the workflow avoids fetching altogether.
Avoiding unnecessary loops	Do not fetch all records and loop when a criterion or direct value can be used.	Followed: the script has no loops.
Built-in functions	Use functions such as ifnull and toString instead of repeated manual processing.	Used in the script.
Indexed fields	May improve retrieval for frequently queried data; can need platform configuration.	Not configured.
Efficient workflows	Keep workflows simple and avoid nested or repeated operations.	General practice.

8. Result

- The workflow Send appoinment confirmation mail was examined in Appointment_Booking_Form.
- Its Deluge script processes only the submitted booking using values from input, with no record fetch and no loops.
- A booking was submitted and the confirmation email was received.
- No benchmark was run, so no execution-time improvement is claimed.

9. Conclusion

Performance in a low-code application depends on how workflows and scripts are written as well as on the platform. In this practical, the confirmation workflow processed only the booking being submitted, handled empty fields with defaults, and sent a single email without loops. Where a record does have to be fetched, a criteria such as `[ID == input.ID]` keeps retrieval to the one record needed. These practices help applications handle growing data and remain maintainable. The benefits are described in terms of reduced data retrieval and processing, not measured execution times.

24101068